

CDA-GNN: A Chain-driven Accelerator for Efficient Asynchronous Graph Neural Network

Hui Yu[†], Yu Zhang[†], Ligang He[§], Donghao He[†], Qikun Li[†], Jin Zhao[†], Xiaofei Liao[†], Hai Jin[†], Lin Gu[†] and Haikun Liu[†]

[†]National Engineering Research Center for Big Data Technology and System

Service Computing Technology and System Lab, Cluster and Grid Computing Lab

School of Computer Science and Technology, Huazhong University of Science and Technology, China

[§] Department of Computer Science, University of Warwick, United Kingdom

{huiy, zhyu}@hust.edu.cn ligang.he@warwick.ac.uk {hdh, lqk2021, zjin, hjin, xfliao, lingu, hkliu}@hust.edu.cn

Abstract

Asynchronous Graph Neural Network (AGNN) has attracted much research attention because it enables faster convergence speed than the synchronous GNN. However, existing software/hardware solutions suffer from *redundant computation overhead* and *excessive off-chip communications* for AGNN due to irregular state propagations along the dependency chains between vertices. This paper proposes a chain-driven asynchronous accelerator, *CDA-GNN*, for efficient AGNN inference. Specifically, *CDA-GNN* proposes a chain-driven asynchronous execution approach into novel accelerator design to regularize the vertex state propagations for fewer redundant computations and off-chip communications and also designs a chain-aware data caching method to improve data locality for AGNN. We have implemented and evaluated *CDA-GNN* on a Xilinx Alveo U280 FPGA card. Compared with the cutting-edge software solutions (i.e., Dorylus and AMP) and hardware solutions (i.e., BlockGNN and FlowGNN), *CDA-GNN* improves the performance of AGNN inference by an average of 1,173x, 182.4x, 10.2x, and 7.9x and saves energy by 2,241x, 242.2x, 12.4x, and 8.9x, respectively.

ACM Reference Format:

Hui Yu, Yu Zhang, Ligang He, Donghao He, Qikun Li, Jin Zhao, Xiaofei Liao, Hai Jin, Lin Gu and Haikun Liu. 2024. CDA-GNN: A

Yu Zhang (zhyu@hust.edu.cn) is the corresponding author of this paper. This paper is supported by National Key Research and Development Program of China (No. 2022YFB2404202), Key Research and Development Program of Hubei Province (No. 2023BAB078), Knowledge Innovation Program of Wuhan-Basi Research (No. 2022013301015177), and Huawei Technologies Co., Ltd (No. YBN2021035018A6).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3656240>

Chain-driven Accelerator for Efficient Asynchronous Graph Neural Network. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3656240>

1 Introduction

Graph Neural Networks (GNNs) have been used in a wide range of real-world applications [10]. The vertices in GNNs follow a neighborhood aggregation scheme, communicating with their neighboring vertices for multiple iterations via synchronous message passing. However, the recent studies [4, 9] show that they suffer from poor performance because of the frequent global synchronization between layers. Thus, a more efficient GNN model called *Asynchronous Graph Neural Network* (AGNN) [2], emerges to address this problem. In contrast to the traditional GNN model, the state (i.e., feature vector) of each vertex in AGNN is asynchronously updated by aggregating and transforming the states of its neighbors until the whole process converges. The gradients and embeddings of the AGNN model have also been proven with the convergence guarantee [2].

Although several software AGNN systems [2, 4, 7, 9] and hardware GNN accelerators [8, 15] have been proposed, they suffer from redundant computations and excessive off-chip communications because of the following two reasons. First, within each iteration of AGNN, the active vertices processed by different cores irregularly propagate the most recent states to the neighbors along the dependency chains¹ between these vertices in an asynchronous way. As a result, most vertices may use the stale states of their neighbors to conduct state updates, which are unnecessary. Second, AGNN also suffers from serious *irregular memory access* because the vertices' states are too large to be stored in the on-chip memory [12] and sparsely dispersed in the off-chip memory.

To address the issues mentioned above, this paper designs a chain-driven hardware accelerator *CDA-GNN* for efficient AGNN inference, which supports an effective chain-driven

¹In AGNN inference, each vertex's state depends on its neighbor's state, because this vertex updates its state through gathering its direct neighbor's state along an edge. It means that the graph path between two vertices will incur a dependency chain between these two vertices' states along this path, because of the dependency between each vertex and its direct neighbor caused by the edge on this path.

asynchronous execution approach to ensure fewer vertex state updates and better data locality. Specifically, in AGNN inference, when the active vertices on a dependency chain are processed consecutively and asynchronously along this dependency chain during an iteration, the other vertices need fewer state updates in this iteration. Thus, *CDA-GNN* tracks the dependency chains between the active vertices on the fly and consecutively processes the active vertices on each tracked chain along their order on this chain. By such means, the data accessing and processing associated with many vertex state updates can be spared. Due to the real-world graphs' power-law property [5], most vertex state propagations occur only on a small set of dependency chains, which indicates that these chains are frequently-used ones. Thus, for better data locality, *CDA-GNN* specializes the memory subsystem to preferentially cache the states on the frequently-used dependency chains in the on-chip memory.

We have implemented and evaluated *CDA-GNN* on a Xilinx Alveo U280 FPGA card. The experimental results show that *CDA-GNN* outperforms the cutting-edge software AGNN solutions (i.e., Dorylus [9] and AMP [2]) running on Intel Xeon CPU and NVIDIA A100 GPU by, on average, 1,173x and 182.4x with 2,241x and 242.2x energy savings, respectively. Compared with the state-of-the-art GNN accelerators, i.e., BlockGNN [15] and FlowGNN [8], *CDA-GNN* achieves the speedup of 8.6x-12.7x and 5.8x-9.7x with 9.4x-21.3x and 6.5x-15.9x energy savings, respectively.

2 Background and Motivation

2.1 Background of AGNN

Existing AGNN models use the popular *Compress Sparse Row* (CSR) [14] format to store the graph and follow an asynchronous message passing scheme [2]. In general, the AGNN adopts the vertex-program abstraction into three phases [2]: *ApplyEdge*, *Aggregation*, and *ApplyVertex* to abstract the dataflow of asynchronous message passing. The *ApplyEdge* phase evaluates each edge's message using neural networks to transform the states of the source vertex and this edge, and the *Aggregation* phase asynchronously gathers the neighbors' states. In the end, the *ApplyVertex* produces a new state for the target vertex using neural networks by combining the previous state of the target vertex and the state gathered in the *Aggregation* phase. Figure 1 gives an example to show the execution of AGNN inference. Each vertex (e.g., v_3) scatters the edge's message (e.g., *Message A*) to its neighbor (e.g., v_2) (①), and the neighbor (i.e., v_2) asynchronously gathers the received message (②) and combines its previous state and the received message to produce a new state (③).

AGNN can generally be classified into two types of execution ways [2]: the ones without *ApplyEdge* and the ones with *ApplyEdge*. The first type, e.g., asynchronous GCN [10], only consists of an *Aggregation* phase and an *ApplyVertex* phase. Compared with the first type, the second type, e.g., asynchronous GIN [11], involves an additional phase (i.e., the *ApplyEdge* phase) before the *Aggregation* phase. Note that

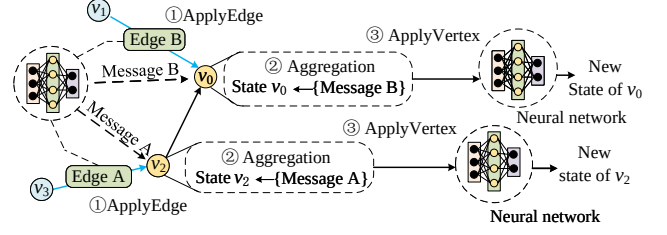


Figure 1. Illustration of the execution of AGNN inference

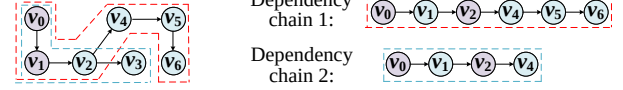


Figure 2. An example how to divide the graph into dependency chain

the *ApplyEdge* and *ApplyVertex* phases can be represented by *Matrix-Vector Multiplication* (MVM).

2.2 Limitation of State-of-the-art Solutions

Many solutions [2, 4, 7, 9] have been proposed to handle the AGNN model on general-purpose processors. However, AGNN inference exhibits *redundant computations* and *excessive off-chip communications* due to its inherent irregular vertex state propagation. Figure 2 gives an example of the execution of AGNN, where the vertices v_0 and v_1 are assumed to be activated in an iteration. When processing v_0 and v_1 in this iteration, v_0 scatters its state to its neighbor v_1 , while v_1 will scatter its state to its direct neighbor v_2 . Nevertheless, v_1 's state needs to be recalculated and updated when v_1 receives the latest state of v_0 . It indicates the state updates of v_1 and its successors (i.e., v_2, v_3, v_4, v_5 , and v_6) are unnecessary.

To prove it, we measure the performance of three cutting-edge software AGNN systems (i.e., GNNAuto [4], Sancus [7], and AMP [2]) running on NVIDIA A100 GPU. Section 4 presents the details of the platform and benchmarks. As shown in Figure 3 (a), AMP outperforms other systems in all cases. However, as illustrated in Figure 3 (b), only 8.3%-10.2% of the total updates performed by AMP are useful in the GIN model. As shown in Figure 3 (b) and (c), the unnecessary off-chip communications take up more than 86.8% of the overall execution time, and accesses to vertex states account for 81.2%-91.4% of all accesses.

Compared with the solutions based on the general-purpose processors, specialized hardware accelerators [8, 12, 15] gain significant energy saving and performance speedup for GNN. However, they are also inefficient for AGNN because they also suffer from significant redundant computations and off-chip communications due to the above described unnecessary updates. In other words, the conventional architectures are also not the best platforms for running AGNN, which requires designing a new hardware accelerator to address the challenges of irregular state propagation for AGNN.

2.3 Motivation

2.3.1 Chain-driven Asynchronous Execution Approach.

To improve AGNN inference's efficiency, we propose a chain-aware asynchronous execution approach. It initiates with

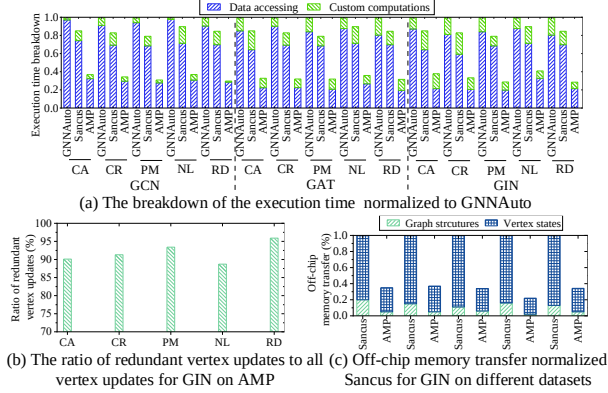


Figure 3. Performance of existing solutions

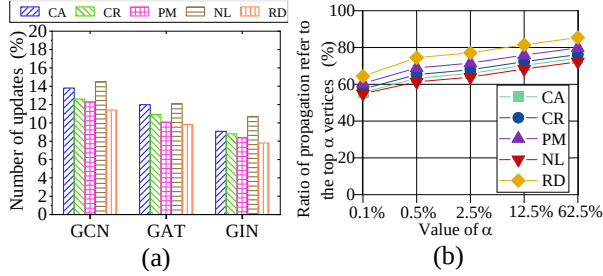


Figure 4. Performance of software approach: (a) the number of updates of our software approach normalized to that of AMP over one GPU; (b) the ratio of propagations passing through the chains between the top α highest degree vertices

an active vertex, employing on-the-fly depth-first traversal throughout each iteration to track dependency chains among active vertices until all active vertices are visited. Then, the active vertices are processed in a sequential and asynchronous manner, adhering to their respective positions within these chains, which can significantly reduce the number of updates because it ensures that the most recent states of active vertices are efficiently propagated to other vertices in the graph.

Take Figure 2 as an example, where the vertices v_0 and v_1 are assumed to be activated in an iteration. There is a dependency chain (i.e., $v_0 \rightarrow v_1$) between the states of v_0 and v_1 . It means that the state propagation originated from v_0 will pass through v_1 . Therefore, in our approach, only when the state propagation originated from v_0 has reached v_1 , and has been gathered by v_1 , v_1 propagates its new state to its neighbors (e.g., v_2) for the state updates of these neighbors. It indicates that the state propagations originated from both v_0 and v_1 can pass through their common successors (e.g., v_2 , v_4 , v_5 , and v_6) together along the dependency chain. In this way, for the state propagations of both v_0 and v_1 , the graph data associated with v_2 , v_4 , v_5 , and v_6 are loaded and processed only once in an iteration, which offers the faster state propagation. Figure 4 (a) shows the vertex state updates needed by our approach is less than 14.5% of that of AMP.

Besides, because of the power-law property [5], as shown in Figure 4 (b), in AGNN, more than 61% of the propagations

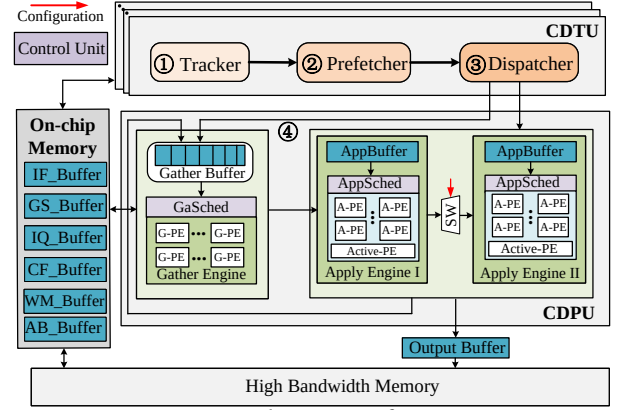


Figure 5. Architecture of CDA-GNN

need to pass through the dependency chains between the top 0.5% highest degree vertices. Thus, our approach also caches the states of the vertices on these frequently-used chains in the on-chip memory for fewer off-chip communications.

2.3.2 Challenges. Although our chain-driven asynchronous execution approach ensures much fewer redundant computations and data accesses, applying our approach into AGNN inference remains challenging, because the runtime overhead occupies 86.3%-96.4% of the total execution time for our tested cases. It is because it needs to take the active vertices as the root to track the dependency chain by irregularly traversing graph on the fly. Besides, it incurs additional instructions and low instruction level parallelism in the general-purpose processors due to the data-dependent branches (which depend on irregular graph structures) of these instructions. It motivates us to design this approach into our hardware accelerator, i.e., *CDA-GNN*, for efficient AGNN inference.

3 CDA-GNN Architecture

3.1 CDA-GNN Overview

Figure 5 shows the architecture of *CDA-GNN*, which is designed to efficiently support the proposed chain-driven asynchronous execution approach to reduce unnecessary computations and off-chip communications for AGNN inference. *CDA-GNN* contains two key hardware units, i.e., *Chain Driven Traveler Unit* (CDTU) and *Chain Driven Processing Unit* (CDPU), and on-chip buffers. The main functionalities of CDTU, CDPU, and on-chip buffers are as follows.

CDTU. CDTU contains three key parts (i.e., the *Tracker*, the *Prefetcher*, and the *Dispatcher*). For efficient tracking of the dependency chains, an *Intermediate_Queue* (i.e., *IQ_Buffer*) is created by the CDTU to store the intermediate information for graph traversal, and a *Chain FIFO* buffer (i.e., *CF_Buffer*) maintains the visited vertices. Meanwhile, the *Active Bitvector* buffer, i.e., *AB_Buffer*, is also created by CDTU to record the active vertices of the graph, avoiding redundant graph exploration. The *Tracker* is used to track the dependency chains between the active vertices' states, and then the *Prefetcher* is responsible for prefetching the graph data on these chains based on the tracked information. Afterward, the *Dispatcher* dispatches the tasks of state updates to the CDPU.

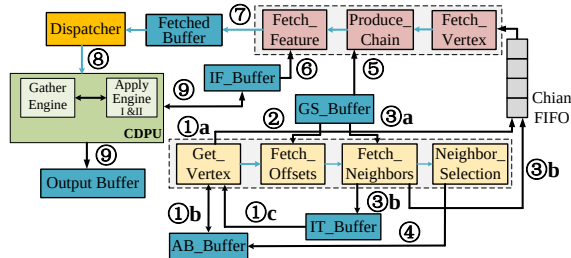


Figure 6. Microarchitecture of CDTU

CDPU. The CDPU is divided into distinct computational modules: a *Gather Engine*, an *Apply Engine I*, and an *Apply Engine II*, each optimized for specific tasks within the AGNN inference process. The *Gather Engine* is used to asynchronously gather the neighbors’ states for each target vertex at the *Aggregation* phase, and the *Apply Engines* are used to perform dense and regular MVM operations at both *ApplyEdge* and *ApplyVertex* phases. For AGNN variants with *ApplyEdge* phase, *Apply Engine I* performs the *ApplyEdge*, while *Apply Engine II* performs *ApplyVertex*. CDA-GNN implements a traditional systolic array architecture for *Multiply-Accumulate* (MAC) computations, while Active-PE manages the activation functions crucial for GNN inference. The *GaSched* of the *Gather Engine* and the *AppSched* of the *Apply Engine* guarantee the pipeline execution and assign the workloads to the *Gather Engine* and the *Apply Engine*, respectively, which works in a task disperse aggregation mode similar to HyGCN [12] for workload balance and task-level parallelism.

On-chip Buffers and Control Unit. The on-chip memory is composed of several buffers, e.g., *GS_Buffer*, *IF_Buffer*, and *Weight_Buffer*, which are employed to cache various data (e.g., *Graph Structure*, *Input Feature*, and *Weight Matrices*) to improve data reuse and reduce unnecessary off-chip communications. To fully exploit the data locality in AGNN inference, it also dynamically caches the states of the vertices on the frequently-used dependency chains, avoiding their data thrashing. Note that *CDA-GNN* adopts the ping-pong buffering technology [12] to hide the access latency.

Workflow of CDA-GNN. CDA-GNN adopts the vertex-centric programming for efficient AGNN inference. In detail, CDA-GNN loads the graph data from off-chip HBM to on-chip memory. The CDTU tracks the dependency chains between the active vertices’ states until all active vertices are visited (the step ❶), and the *Prefetcher* takes each active vertex to prefetch the edges of this vertex and the states of both source/target vertices of these edges (the step ❷). These graph data of the prefetched edges are resident in the *Chain FIFO* buffer. After that, the *Dispatcher* dispatches the tasks of neural networks to the *AppBuffer* of the *Apply Engine* and the aggregation tasks to the *Gather Buffer* of the *Gather Engine*, respectively, for updating the target vertices’ states (the step ❸). If a task exists in the *Gather Buffer* or the *AppBuffer*, CDA-GNN will trigger the *ApplyEdge*, the *Aggregation*, and *ApplyVertex* operations in parallel for target vertices until these buffers are empty (the step ❹).

Table 1. Dataset Information

Datasets	#Vertices	#Edges	Feature length
Cora (CA)[4]	2,708	10,556	1,433
Citeseer (CR)[5]	3,327	9,104	3,703
Pubmed (PM)[6]	19,717	88,648	500
Nell (NL)[7]	65,755	266,144	61,278
Reddit (RD)[8]	232,965	114,615,892	602

Table 2. Resource Utilization on Xilinx Alveo U280 FPGA

Resource	GCN	GIN	GAT
DSP	24.3%	17.5%	21.4%
LUT	34.7%	26.3%	24.2%
FF	31.4%	29.4%	39.1%
BRAM	42.1%	46.2%	44.5%

3.2 Chain-aware Regularized Data Accessing

Dependency Chain Tracking. To efficiently track the dependency chain for fast state propagation, within each iteration, the *Tracker* takes the unvisited active vertices as the roots to track the chains in a depth-first fashion on the fly until all vertices have been visited. As shown in Figure 6, the *Tracker* has four stages, which are implemented as a pipeline. At the *Get_Ver* stage, if the *IQ_Buffer* is empty, the *Tracker* scans the *AB_Buffer* to find an active vertex (1a), then marks this vertex as inactive intermediately for the correctness and inserts this found vertex into the *CF_Buffer* (1b). If not, the *Tracker* fetches a vertex from the *IQ_Buffer* (1c). After that, the *Fetch_Offsets* stage obtains the first and last offsets of the vertex from the *Vertice* array of the *GS_Buffer* (2). The *Fetch_Neighbors* stage loads the unvisited neighbors of this vertex from the *Neighbor* array of the *GS_Buffer* (3a), and these neighbors are pushed into the *IQ_Buffer* and *CF_Buffer* (3b). At the *Neighbors_Selection* stage, these fetched neighbors are marked as inactive if these neighbors are active in the *AB_Buffer* (4). The above procedures repeat until all active vertices have been visited. The sequence of visited vertices between two active vertices, which is resided in *CF_Buffer*, is approximately regarded as the dependency chain between the active vertices' states.

Graph Data Prefetching. To efficiently prefetch graph data on the dependency chains, the *Prefetcher* manages a pipeline with three stages, i.e., *Fetch_Verx*, *Produce_Chain*, and *Fetch_Feature* stage. At the *Fetch_Verx* stage, the *Prefetcher* fetches a vertex (i.e., the source vertex ID) from the *CF_Buffer* sequentially. The *Produce_Chain* stage fetches this vertex's neighbors (i.e., the target vertices ID) from the *Edges* array to produce the chain (⑤). At the *Fetch_Feature* stage, for each edge of this chain, the source vertex state, the target vertex state, and the edge state are accessed from the *IF_Buffer* (⑥). The prefetched edges, the state of edges, and the vertices' states associated with these edges are pushed into the *Fetched Buffer* (⑦). Note that each entry in *Fetched Buffer* takes the form of *<Source ID, Target ID, Source vertex state, Target vertex state, Edge state>*.

3.3 Chain-driven Asynchronous Processing

To significantly reduce the end-to-end inference latency of AGNNs, the CDPU orchestrates the operations of *Aggregation*, *ApplyVertex*, and *ApplyEdge* in an asynchronous

pipelined fashion for each vertex along the vertex chain. For each target vertex, the *Dispatcher* simultaneously assigns *Aggregation* tasks to the *Gather Engine* and *ApplyEdge* tasks to *Apply Engine II*. Once the *Gather Engine* and *Apply Engine II* complete their computations, the results are immediately dispatched to *Apply Engine I* for the final state computation. Specifically, *Dispatcher* transfers the states of source and target vertices from the *Fetch_Buffer* to the *Gather Buffer* of the *Gather Engine*. Simultaneously, it ensures the transfer of the state of the source vertex, along with the corresponding edge state to the *AppBuffer* of *Apply Engine II*.

Note that for the AGNN variants without *ApplyEdge* operation (e.g., GCN model [10]), CDPU via the control unit and a hardware *Switch* (i.e., SW), merges *Apply Engine I* and II into a singular larger *Apply Engine*. This merger effectively executes the *ApplyVertex* operation, optimizing hardware resource utilization and enhancing the efficiency of computational units. Specifically, this configuration is controlled by a 1-bit configuration word for the SW: '0' indicating independent operation of *Apply Engine I* and II, and '1' signifying their merger for *ApplyVertex* tasks.

To circumvent pipeline stalling associated with *Aggregation* operations, a unique G-PE design has been proposed. Each G-PE within the *Gather Engine* contains 8 parallel aggregation modules. Each module is equipped with two input queues, *Input Queue 1* for storing source vertex states and *Input Queue 2* for the target vertex states. These are processed through an adder followed by a multiplexer, determining whether to output the result or cache it as an intermediate value in the *Partial_Buffer*. Besides, the *Gather Engine* adopts feature-level parallelism to execute *Aggregation*.

3.4 Chain-aware Data Caching

For efficient access of the *Input Feature*, *Graph Structure*, *Intermediate_Queue*, *Chain FIFO* buffer, *Weight Matrices*, and *Active Bitvector*, as shown in Figure 5, six on-chip buffers, i.e., *IF_Buffer*, *GS_Buffer*, *IQ_Buffer*, *CF_Buffer*, *WM_Buffer*, and *AB_Buffer*, are used to cache these data, respectively. To further reduce unnecessary off-chip communications, *CDA-GNN* designs a *Chain-Aware Data Caching* (CADC) scheme to manage the vertex states cached in the on-chip *IF_Buffer* because the off-chip communication of vertex states dominates the overall performance of AGNN inference. Note that the other buffers are managed using LRU [6] by default. When the states of a vertex v are requested, if the *IF_Buffer* is not full, v 's states will be directly cached in the *IF_Buffer*. Otherwise, CADC first obtains v 's priority, i.e., $Pri(v) = \frac{P_G(v)}{P_G} \times D_v$, where P_G denotes the number of chains in graph G , $P_G(v)$ represents the number of chains that include v as a critical vertex and can be calculated by CDTU, and v 's degree D_v (i.e., subtracting v 's beginning offset from its end offset) are obtained according to the *Vertices* array.

Once the priority score is computed, CADC compares v 's priority with those of the vertices currently residing in the *IF_Buffer*. If v has a higher priority than the lowest-priority

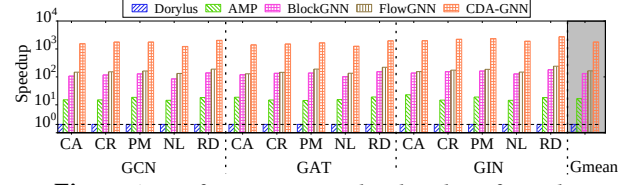


Figure 7. Performance normalized to that of Dorylus

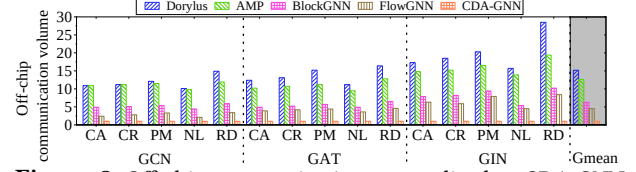


Figure 8. Off-chip communications normalized to CDA-GNN

vertex in the buffer, CADC will replace the lower-priority vertex with v .

4 EVALUATION

4.1 Experiment Setup

CDA-GNN Settings. We have implemented *CDA-GNN* on a Xilinx Alveo U280 FPGA card, which is equipped with a XCU280 FPGA chip. The FPGA provides 9 MB BRAM resources, 1.3 M LUTs, 2.6 M Registers, and two 4 GB HBM2 stacks. *CDA-GNN* contains 8 CDTUs and the on-chip memory of *CDA-GNN* is implemented using the BRAM resources. Similar to BlockGNN [15], for the PE Group of *CDA-GNN*, we adopt programmable ALUs implement different operations depending on the properties of the AGNN model. *Activate PEs* support activation operations, while the PEs in the *Apply Engines* are constructed as a systolic array. Each PE is a MAC unit with some glue logic. Note that we configured two systolic modules (each module has 32×128 arrays) and 512 G-PEs of the *Gather Engine* for the *CDA-GNN*. We get the clock rate of *CDA-GNN* using Xilinx Vivado 2019.1 and conservatively use 330 MHz in our experiments. The resource utilization of all models is reported in Table 2.

Datasets and Benchmarks. We use five datasets (listed in Table 1) and three GNN models using asynchronous message passing, i.e., GCN [12], GIN [7], and GAT [10], which are commonly used in GNN acceleration studies [15].

Baseline. The performance of *CDA-GNN* is compared with four solutions, i.e., Dorylus [9], AMP [2], BlockGNN [15], and FlowGNN [8]. Dorylus runs on the CPU platform (which has an Intel Xeon 6151 processor with 65 cores at 3.0 GHz and 696 GB DRAM), and AMP runs on the NVIDIA Tesla A100 with 6,912 cores and 80 GB HBM. Besides, *CDA-GNN* is also compared with the cutting-edge GNN accelerators, i.e., BlockGNN and FlowGNN, which run at 330 MHz on the Xilinx Alveo U280 FPGA.

4.2 Experimental Results

4.2.1 Overall Performance. Figure 7 shows the execution time of different solutions normalized to that of Dorylus. Compared with Dorylus, AMP, BlockGNN, and FlowGNN, *CDA-GNN* improves the performance by 1,055x-1,834x (1,173x on average), 142.2x-282.8x (182.4x on average), 8.6x-12.7x (10.2x on average), and 5.8x-9.7x (7.9x on average), respectively. The better performance achieved by *CDA-GNN* comes

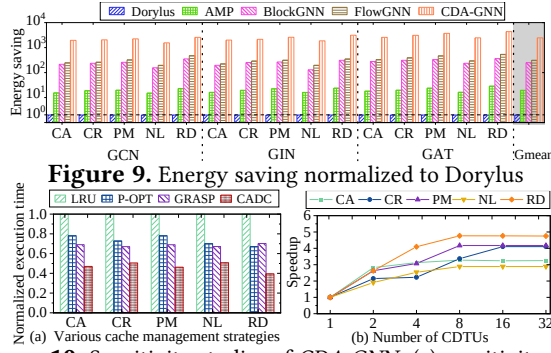


Figure 10. Sensitivity studies of *CDA-GNN*: (a) sensitivity to the replacement strategies; (b) sensitivity to the number of CDTUs from fewer redundant computations and off-chip communications. Note that our CADG strategy contributes 5.1%-8.4% of *CDA-GNN*'s performance improvement. For our tested instances, the number of vertex state updates of *CDA-GNN* is only 7.8%-14.5% of that of them. It is because the new states of vertices can be propagated along the dependency chains more efficiently in *CDA-GNN* than them. It also indicates that *CDA-GNN* can spare data access costs associated with these redundant updates. Thus, as shown in Figure 8, for the GIN model on RD, the amount of off-chip communications of Dorylus, AMP, BlockGNN, and FlowGNN are 28.5x, 19.4x, 10.2x, and 8.4x more than that of *CDA-GNN* on average, respectively, because of much more updates and poorer data locality when accessing vertex states. Figure 9 shows that the energy savings of *CDA-GNN* are 1,785x-2,815x (2,241x on average) and 186.4x-321.4x (242.2x on average) are higher than Dorylus and AMP, respectively. Compared with BlockGNN and FlowGNN, the energy savings of *CDA-GNN* are improved by an average of 12.4x and 8.9x.

4.2.2 Sensitivity Studies. Figure 10 (a) shows the performance of *CDA-GNN* with different replacement strategies (i.e., LRU [6], P-OPT [1], and GRASP [3]) for the management of the *IF Buffer* on asynchronous GIN model. It shows that our CADG scheme outperforms the others because states of the vertices on these frequently-used chains can be effectively cached in the on-chip memory. Figure 10 (b) describes the sensitivity to the number of CDTUs. It shows that *CDA-GNN* obtains better performance as the number of CDTUs increases until the number of CDTUs reaches eight because the memory bandwidth is saturated.

5 Related Work

Software AGNN Systems and GNN Accelerators. Dorylus [9] is the first system to support AGNN model. AMP [2] proposes a GPU-based approach for better performance of AGNN. Besides, to accelerate the GNN inference, FlowGNN [8] uses an approach based on the multi-queue streaming, while BlockGNN [15] proposes a pipelined architecture to support block-circulant matrices computation. However, these solutions suffer from irregular state propagation for AGNN.

Asynchronous Graph Processing Solutions. Maiter [13] proposes a delta-accumulative computation approach. AsyncGraph [14] further extends this approach to support GPU.

However, they can not support the neural network operation of AGNN inference [2].

6 Conclusion

This paper proposes a chain-driven hardware accelerator *CDA-GNN* for efficient AGNN inference. *CDA-GNN* regularizes the state propagations along dependency chains and efficiently cache the states of frequent-used vertices on dependency chains. Our evaluation demonstrates that *CDA-GNN* outperforms the cutting-edge AGNN solutions significantly.

References

- [1] Vignesh Balaji, Neal Crago, Aamer Jaleel, and Brandon Lucia. 2021. P-OPT: Practical Optimal Cache Replacement for Graph Analytics. In *Proceedings of HPCA*. 668–681.
- [2] Lukas Faber and Roger Wattenhofer. 2023. GwAC: GNNs with Asynchronous Communication. In *Proceedings of LoG*. 1–20.
- [3] Priyank Faldu, Jeff Diamond, and Boris Grot. 2020. Domain-Specialized Cache Management for Graph Analytics. In *Proceedings of HPCA*. 234–248.
- [4] Matthias Fey, Jan Eric Lenssen, Frank Weichert, and Jure Leskovec. 2021. GNNAutoScale: Scalable and Expressive Graph Neural Networks via Historical Embeddings. In *Proceedings of ICML*. 3294–3304.
- [5] Joseph E. Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. PowerGraph: Distributed Graph-Parallel Computation on Natural Graphs. In *Proceedings of OSDI*. 17–30.
- [6] Daniel A. Jiménez. 2013. Insertion and promotion for tree-based PseudoLRU last-level caches. In *Proceedings of MICRO*. 284–296.
- [7] Jingshu Peng, Zhao Chen, Yingxia Shao, Yanyan Shen, Lei Chen, and Jiannong Cao. 2022. SANCUS: Staleness-Aware Communication-Avoiding Full-Graph Decentralized Training in Large-Scale Graph Neural Networks. *Proceedings of the VLDB Endowment* 15, 9 (2022), 1937–1950.
- [8] Rishov Sarkar, Stefan Abi-Karam, Yuqi He, Lakshmi Sathidevi, and Cong Hao. 2023. FlowGNN: A Dataflow Architecture for Real-Time Workload-Agnostic Graph Neural Network Inference. In *Proceedings of HPCA*. 1099–1112.
- [9] John Thorpe, Yifan Qiao, Jonathan Eyolfson, Shen Teng, Guanzhou Hu, Zhihao Jia, Jinliang Wei, Keval Vora, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2021. Dorylus: Affordable, Scalable, and Accurate GNN Training with Distributed CPU Servers and Serverless Threads. In *Proceedings of OSDI*. 495–514.
- [10] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (2021), 4–24.
- [11] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *Proceedings of ICLR*. 1–17.
- [12] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. 2020. HyGCN: A GCN Accelerator with Hybrid Architecture. In *Proceedings of HPCA*. 15–29.
- [13] Yanfeng Zhang, Qixin Gao, Lixin Gao, and Cuirong Wang. 2014. Maiter: An Asynchronous Graph Processing Framework for Delta-Based Accumulative Iterative Computation. *IEEE Transactions on Parallel and Distributed Systems* 25, 8 (2014), 2091–2100.
- [14] Yu Zhang, Xiaofei Liao, Lin Gu, Hai Jin, Kan Hu, Haikun Liu, and Bingsheng He. 2020. AsyncGraph: Maximizing Data Parallelism for Efficient Iterative Graph Processing on GPUs. *ACM Transactions on Architecture and Code Optimization* 17, 4 (2020), 29:1–29:21.
- [15] Zhe Zhou, Bizhao Shi, Zhe Zhang, Yijin Guan, Guangyu Sun, and Guojie Luo. 2021. BlockGNN: Towards Efficient GNN Acceleration Using Block-Circulant Weight Matrices. In *Proceedings of DAC*. 1009–1014.

Towards Cost-Effective Real-Time High-Throughput End Station Design for Time-Sensitive Networking (TSN)

Chuanyu Xue
University of Connecticut
chuanyu.xue@uconn.edu

Tianyu Zhang
University of Connecticut
tianyu.zhang@uconn.edu

Song Han
University of Connecticut
song.han@uconn.edu

Abstract

Time-Sensitive Networking (TSN) technology has been increasingly deployed in mission- and safety-critical industrial applications to achieve high throughput and deterministic communications. To provide stringent timing guarantee, TSN requires that network devices follow a predefined communication schedule for real-time end-to-end packet processing, involving both TSN bridges and end stations. Extensive efforts have been devoted on the TSN bridge design in the literature. Achieving TSN compatibility on the end stations (especially on COTS hardware), however is challenging due to their constrained resources. To fill this gap, this work presents a software-based open-source TSN end station design that i) enables ultra-low latency and nanosecond-level transmission accuracy based on DPDK, and ii) employs a novel multi-core scheduling framework to boost the throughput of real-time TSN traffic. Our proposed solution leverages existing COTS hardware and thus is more generic and cost-effective compared to existing hardware-centric solutions. We validate our design by developing a prototype end station and incorporating it in a real-world TSN testbed. Our extensive experiments demonstrate the efficiency and effectiveness of our design compared with other state-of-the-art solutions.

1 Introduction

Time-Sensitive Networking (TSN) has been increasingly adopted in industrial Internet-of-Things (IIoT) systems to provide real-time communications for industrial applications [1]. As a set of open standards extended from Ethernet, TSN aims to provide both deterministic communications and much higher throughput compared with traditional fieldbus [2]. To enable stringent real-time communications, TSN standard requires that all the network devices, including both TSN bridges and end stations (ES) [3], are able to support precise packet forwarding according to the pre-defined schedule.

Many commercial TSN bridge products (e.g., TTTech Evaluation Board [4] and Cisco Industrial Ethernet 4000 Switch [5]) can support real-time and high-throughput (e.g., 1 Gbps) traffic with microseconds-level precision. However, due to the inefficiency of general CPU and unpredictable bus contention [6], the design of real-time TSN-compatible ES is much more challenging and remains an open problem. In the literature, real-time TSN ES designs can be classi-

fied into *hardware-based* design and *software-based* design. The former approaches (e.g., [6–9]) implement TSN ES on a dedicated hardware platform, e.g., Field Programmable Gate Arrays (FPGA) and system-on-chip (SoC), which can generally support higher throughput and faster scheduling actions. However, hardware-based solutions are expensive and inflexible. They require long development period and introduce operational complexity in production environment. The software-based solutions (e.g., [10–12]), which implement the TSN ES based on general Linux kernel, are more cost-effective and flexible. However, they suffer from several limitations, including high latency and low throughput, based on our measurement results from a real-world TSN testbed.

In this work, we address these limitations by presenting a software-based open-source approach based on Data Plane Development Kit (DPDK) [13] to design real-time and high-throughput TSN ES on COTS hardware. Our solution enables nanosecond-level packet transmission accuracy on DPDK by offloading the scheduling duty from CPU to network interface card (NIC), and adapting high precision event timer (HPET) to reduce CPU resource contention. To improve the throughput, we propose a novel multi-core stream allocation framework which guarantees that the traffic flows allocated on the same CPU core do not conflict with others.

We validate our design by developing a prototype TSN ES and incorporating it into a real-world TSN testbed. The evaluation on the testbed shows that our proposed solution can support periodic traffic up to 50 kHz with negligible jitter (16 ns) and 0% packet loss ratio. The transmission delay is at least 2 times lower (32 μ s in the worst case) and 8.7 μ s on average compared to existing software-based solutions. Through extensive simulation-based evaluation, we further demonstrate that the proposed multi-core stream allocation framework can achieve 2.4 times higher feasibility ratio compared with that on a single-core platform. Our contributions of this work are summarized as follows:

- 1) We propose a software-based approach based on COTS hardware and open-source DPDK library to design and implement TSN ES with nanosecond-level packet transmission accuracy and high throughput.
- 2) We propose a novel multi-core stream allocation framework with conflict-free guarantees to improve the throughput.
- 3) We perform both hardware-based validation and simulation-based evaluation to demonstrate superior performance compared to other state-of-the-art solutions.

To the best of our knowledge, this is the first work on the design and implementation of DPDK-based TSN ES to support deterministic real-time communications in nanoseconds-level accuracy. Notably, the hardware components of the prototype, including an embedded computation module, IO board, and a commercial NIC have a total cost of less than \$500, underlining the affordability of our approach.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the Owner/Author.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06
<https://doi.org/10.1145/3649329.3658260>

```

- 82.14% __sys_sendmsg
| - 55.85% ____sys_sendmsg
|   - 50.40% sock_sendmsg
|     - 49.93% inet_sendmsg
|       - 49.04% udp_sendmsg
|         - 23.67% udp_send_skb
|           - 22.92% ip_send_skb
|             - 19.11% ip_make_skb
|       - 2.39% uaccess_ttbr0_disable
|       - 2.33% uaccess_ttbr0_enable
| - 25.82% sendmsg_copy_msghdr
|   - 17.66% __copy_msghdr_from_user
|     - 9.53% move_addr_to_kernel.part.0
|     - 4.09% uaccess_ttbr0_disable
|     - 3.65% uaccess_ttbr0_enable

```

Figure 1. The profiling results of Linux API-based TSN ES by Perf.

2 Background and Motivation

While Linux API-based solutions are widely used in TSN ES software implementations, their efficiency of managing real-time TSN traffic is increasingly challenged. For example, [10] implements a software-based solution based on Linux socket with `SO_TXTIME` extension. The authors claim that the proposed solution can support time-triggered (TT) traffic with a period of $31.25 \mu\text{s}$ and worst-case delay of $4.17 \mu\text{s}$. [11] follows a similar design approach to support $100 \mu\text{s}$ periodic traffic with $42 \mu\text{s}$ delay. Both approaches fall short of meeting the industry’s requirements from emerging applications in terms of both transmission frequency and latency, for example, the automation servo control application in TSN profile [14] requires to communicate every $31.65 \mu\text{s}$ in control loop.

In order to gain a quantitative understanding on the limitations of the existing software-based approaches, we conducted an empirical analysis using a Linux API-based ES, drawing on methodologies outlined in [11]. The performance was profiled using *Perf* [15], and the results are presented in Fig. 1. From the results, we can observe that the performance bottleneck is at transferring the packets between user space and kernel space, accounting for 25.82% of runtime. In addition, the processing overheads associated with the Linux network stack, such as IP header preparation, also contribute substantially (19.11%). Beyond these delays, the Linux API-based solution also exhibits the lack of flexibility in handling multi-core parallelism. The scheduler, which is typically integrated with the Linux traffic control module and embedded within the kernel, further complicates CPU pinning and the effects of OS scheduling policy.

To reduce the processing delay in the kernel and increase the flexibility of handling multi-core parallelism, our approach designs the TSN ES using DPDK [13]. DPDK is a set of open-source libraries and drivers developed by Intel for fast packet processing in data plane applications, which is widely used in high-speed network interfaces in recent years. The key principle of DPDK lies in its user space operations, eliminating the need for memory copying as bypassing the kernel’s networking stack, thereby achieving low-latency packet processing. Moreover, DPDK allows the user to directly access the packet descriptor and hardware registers, which is essential for us to implement the high-accuracy and high-throughput TSN ES without relying on kernel functions.

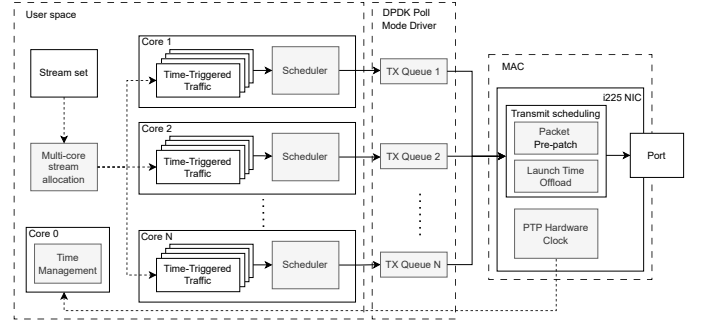


Figure 2. The overall architecture of our TSN ES design based on DPDK.

3 TSN End-Station Design

In this section, we first describe the overall design of our DPDK-enabled TSN ES. We then introduce the key features of each component and the optimization we made to improve the scheduling accuracy and device throughput.

3.1 Overall End Station (ES) Design

The overall architecture of our TSN ES design based on DPDK library is shown in Fig. 2. Our ES design is fundamentally structured two components: the high-precision traffic scheduler and the multi-core stream allocation algorithm. The stream allocation algorithm first determines the specific core responsible for handling each traffic. Subsequently, individual schedulers operating on each core take charge, executing transmissions according to the predefined schedule. To achieve high precision transmission on TT traffic, we first design the scheduler on single core by i) incorporating launch time offloading feature of NIC, and ii) adapting the high precision event timer in the scheduler. To further improve the throughput, we design a multi-core parallel framework for packet processing by leveraging multiple CPU cores on the ES, and utilizing the data pre-fetch hardware function to guarantee the correct traffic order among cores. A parallel contention-free stream allocation algorithm is also designed to optimally assign subsets of streams to specific CPU cores, ensuring that the real-time performance of each traffic type is reliably maintained. We now present the design details below.

3.2 High-Precision Scheduler

The high-precision scheduler aims to send TT traffic following the pre-defined schedule. That is, each packet must be dispatched from the NIC at the exact time specified by the schedule. To achieve this, we first order the streams into an array of packets according to their scheduled transmission time within the hyper-period (the least common multiple of all the stream periods). Then, the scheduler sends the packets in the array following a wake/sleep pattern. Specifically, by comparing the current time with the scheduled time of the packet, the scheduler determines whether to sleep or wake up and send the packet.

To implement the high-precision scheduler, we need to tackle the following two challenges: i) How to achieve deterministic and precise transmission time with minimum jitter?

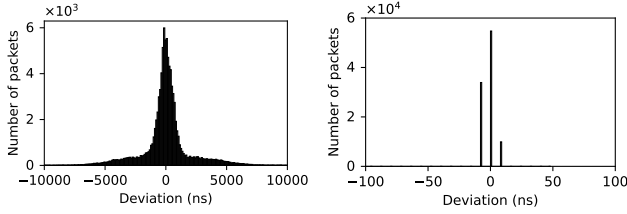


Figure 3. The distribution of time deviation between actual and scheduled sending times under CPU scheduling (left without launch time offload) and NIC scheduling (right with launch time offload) measured in our testbed.

ii) How to avoid transmission congestion and packet loss with high transmission frequency? We discuss our solutions below.

Launch time offload. Traditional TSN ES schedulers rely on CPU scheduling, which suffers from non-deterministic delays caused by command execution time on CPU and direct memory access (DMA) delay. To address this issue, we propose using the hardware offload function to implement the scheduling function on NIC. This is supported by the recent trend of *scheduling time offload* feature development in some NICs (e.g., Intel i210/i225 and Nivida Mellanox CX-5), allowing the NIC to control packet release at specific times. We implement this function in our scheduler based on the latest DPDK (23.07 update), which exposes this offload feature by adding the `SEND_ON_TIMESTAMP` flag in device configure. Once this feature is enabled, we can set the launch time of each packet in the packet advanced descriptor when we send it from the user space.

The main benefit of this design is to isolate the jitter caused by CPU and the random DMA delay between CPU and NIC. Furthermore, as the launch time is referred on the hardware clock of NIC, it also reduces the synchronization error by referring to a more accurate clock. To demonstrate its effectiveness, we compare the jitter distribution of packet transmissions with and without¹ launch time offload in Fig. 3. In the comparison, we send 100,000 packets every 1 ms from the talker and measure the actual time when the packets are received at the listener. We can observe that the delay of sending a packet with launch time offload is much more stable (within 16 ns) than that without launch time offload.

High precision event timer. The second challenge in the implementation of the high-precision scheduler is how to ensure the scheduler runs steadily in high frequency to avoid congestion and packet loss. During our implementation, we identified that the bottleneck is usually caused by the resource contention due to the low timer accuracy. Specifically, if the timer sleeps longer than expected, the time when the CPU issues the packet sending command is already later than the scheduled time, causing the packet miss the sending time. On the other hand, if the timer sleeps shorter than expected, the sending speed will be faster than scheduled and eventually cause the overflow of TX queue, causing the packet loss.

¹For scheduler without launch time offload, the transmission time is controlled by CPU, i.e., a thread sleeps to the scheduled transmission time and call DPDK `rte_eth_tx_burst` function immediately.

Table 1. The CPU cyclic profiling results of nanosleep and HPETsleep.

Cycle	nanosleep			HPETsleep		
	min	mean	max	min	mean	max
1 μ s	2.4 μ s	52.5 μ s	86.7 μ s	94 ns	170 ns	3.5 μ s
10 μ s	2.4 μ s	52.8 μ s	141.3 μ s	91 ns	160 ns	5.5 μ s
100 μ s	2.7 μ s	54.5 μ s	348.5 μ s	92 ns	246 ns	10.3 μ s
1 ms	4.4 μ s	98.7 μ s	820.8 μ s	101 ns	520 ns	10.9 μ s

One straightforward way to design the timer is using nanosleep system call to sleep the thread until the next sending time. However, the nanosleep function is not deterministic and its actual sleeping duration is usually longer than expected according to our experiments. The main reason is that as a system call, the nanosleep function is subject to the OS scheduling policy and it only guarantees the sleeping time longer than the designated time. To solve this issue, we propose to use high precision event timer (HPET) to replace the nanosleep function. HPET is a hardware timer which commonly available in modern computer architecture. It produces periodic interrupts at a much higher resolution than the system RTC clock. In our implementation, the timer resolution is measured as 2.6 GHz on the testbed.

To demonstrate the effectiveness of using HPET timer, we compare it with the precision of the nanosleep function in Table 1. The measurement process is inspired by the well-known *Cyclicttest*, which measures the difference between a thread's intended wake-up time and the time when it actually wakes up in order to provide statistics about the system's latency. We conduct the measurement by using *nanosleep* and HPET timers to sleep the thread in 1 μ s to 1 ms frequency for 5 minutes and measure the actual sleeping time. Both measurements are referred to the physical clock resident in NIC for fair comparison. From the results, we can observe that the HPET timer can achieve much better worst-case precision (up to 10.9 μ s) than the *nanosleep* function. In addition, we find that the HPET timer can achieve sub-microsecond precision on average.

3.3 Multi-Core Parallel Transmission

Due to the non-deterministic delays introduced by the CPU and DMA access, we have to insert a safe inter-space between any two consecutive sending commands on each CPU core to compensate for the worst-case combined delay. Otherwise, the accumulated jitter may cause the packets miss their sending times violating the deterministic transmission requirement. Such inter-space guide time significantly limits the network throughput.

To improve the throughput, we propose to rely on the multi-core architecture by leveraging DPDK's multi-process feature. The key idea is to divide the stream set into multiple subsets and allocate each stream subset on a specific CPU core. DPDK supports the lock-free operation on TX queue connected to each CPU core and this allows the scheduler on different CPUs to send packets concurrently without any conflict. As shown in Fig. 2, we use the master core (core 0) to manage the synchronization and allocate memory buffer

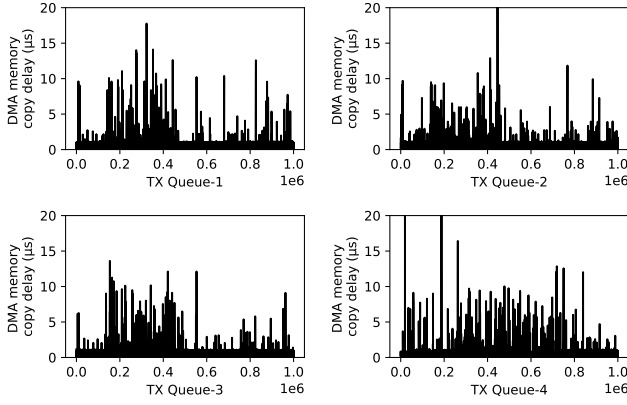


Figure 4. The DMA delay measurement of different TX queues. The X-axis represents the packet index during measurement.

pool and TX queue for each slave core. Each TX queue is associated with one scheduler for the lock-free operation.

However, a major challenge under the multicore parallel transmission design is how to guarantee the packet sending order across different CPUs and the corresponding TX queues given NIC's default priority-based fetch algorithm. Specifically, the packets from the TX queue with higher priority will be fetched first, regardless of their scheduled transmission times. To tackle this issue, we manage to use the data pre-fetch offload function in NIC to guarantee that the packet always be fetched at specific time in NIC.

Data pre-fetch is a hardware-based offloading function that allows the NIC to fetch packets from the memory before launch time. CPU will inform NIC at what time to retrieve packet when it writes TX queue. By enabling this function, we can make sure the packet always pre-fetched at NIC before the launch time comes. Since the data pre-fetch function is not supported by the latest DPDK version yet, we modify the DPDK igc driver to enable this function by writing `DATA_FETCH_TIM` and `FETCH_TIM_DELTA` register fields. Furthermore, since the data pre-fetch function fetches the packet from descriptor to NIC's internal buffer prior to launch time, we need to decide this time distance. We measured the DMA delay of different TX queues in our testbed using the `dpdk-test-dma-perf` tool. According to the measurement results in Fig. 4, the DMA access delay is less than $5 \mu s$ on average but reaches $20 \mu s$ in the worst case. Thus, we set the data pre-fetch time to $50 \mu s$ in our implementation.

Fig. 5 summarizes the operation sequence enabled with both data pre-fetch and launch time offload for packet transmission.

4 Multi-Core Stream Allocation

In our multi-core parallel transmission design, we need to guarantee that the traffic streams allocated to the same CPU core do not conflict. To achieve this, in this section, we first present the system model and formulate the multi-core contention-free stream allocation problem. Then, we map the problem to a classical graph k -coloring problem, and leverage

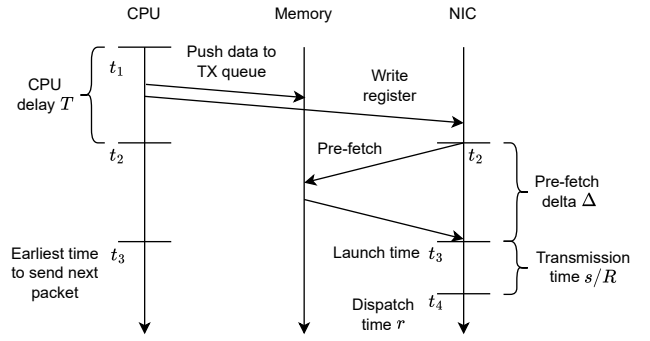


Figure 5. The operation sequence of transmitting a packet in our scheduler design: the CPU issues the sending command at t_1 and the NIC fully dispatches the packet on wire at t_4 .

the existing graph coloring algorithms to determine the core allocation for the deterministic TT traffic.

4.1 System Model

Our TSN ES design can be formalized as $E = \{R, Q, \Delta, T\}$ with the line rate R of NIC, the number of available TX queues Q , the pre-fetch time Δ , and the worst-case CPU delay T . Each TX queue is associated with a dedicated CPU core $c_i \in C$. On each CPU core, we assume that the timing overhead caused by synchronization error and command execution time is bounded by a constant T . Each core is responsible for handling a set of TT traffic $f \in F$ following the network-wide TSN schedule. We formalize the schedule of each TT traffic as $f_i = \{p_i, r_i, s_i\}$ with period p_i , predefined releasing offset from NIC r_i , and frame size s_i . T_{hyper} represents the hyper-period of stream set.

4.2 Problem Formulation

The objective of the multi-core stream allocation problem is to find a stream-to-CPU mapping $g : F \rightarrow C$ that guarantees all streams scheduled on the same CPU core do not conflict. Specifically, the transmission conflict occurs if: 1) the CPU time occupied by two packets on the same core are overlapped, 2) the number of packets in the same queue exceeds the queue size at any time. In this paper, we simplify the problem by imposing a stricter constraint, limiting the TX queue size to one. This approach results in a sub-optimal yet safe allocation. The problem is formalized with following constraint:

$$\begin{aligned} & \forall f_i, f_j \in F, i < j, g(f_i) = g(f_j) \\ & \forall k_i \in T_{hyper}/p_i, \forall k_j \in T_{hyper}/p_j \Rightarrow \\ & (r_i - (s_i/R + \Delta + T))k_i p_i \geq (r_j - s_j/R)k_j p_j \quad \text{Or} \\ & (r_j - (s_j/R + \Delta + T))k_j p_j \geq (r_i - s_i/R)k_i p_i \end{aligned} \quad (1)$$

Fig. 5 shows an example. Given the transmission time of each stream at t_4 , we aim to allocate this stream to CPU cores and make sure there is no other stream overlapping in the preparing time (t_1 to t_3) on the same CPU core.

Next, we translate the multi-core stream allocation algorithm to classical graph coloring problem. We first show how to construct a conflict graph representation of the original problem, where each vertex corresponds to a TT traffic

stream and edges denote the scheduling conflicts. We then apply graph k -coloring algorithm to find a contention-free allocation from streams to CPU cores. We first give two key definitions of the conflict graph:

Definition 1. *There exists a scheduling conflict between f_i and f_j if and only if it violates constraint (1) when f_i and f_j are allocated on the same CPU core.*

Definition 2. *A conflict graph $G = (V, E)$ is a graph representation of the original multi-core stream allocation problem, where each vertex $v_i \in V$ corresponds to the schedule of TT traffic $f_i \in F$, and each edge $e_{i,j} \in E$ denotes the scheduling conflict between f_i and f_j .*

We now show that the multi-core stream allocation problem can be solved by finding a k -coloring of the conflict graph G .

Theorem 1. *Let $G = (V, E)$ be a conflict graph of the original scheduling problem. There exists a feasible mapping $g : F \rightarrow C$ if G is a k -colorable graph that $k \leq |Q|$.*

Proof. Assume G is a k -colorable graph where $k \leq |Q|$. This means that the vertices of G can be colored with at most Q colors such that no two adjacent vertices (representing streams with scheduling conflicts) share the same color. Each color used in this graph coloring can be associated with a distinct CPU core. Since adjacent vertices cannot share the same color, this ensures that no two conflicting streams are allocated to the same core. Therefore, a valid k -coloring of G directly translates to a valid mapping $g : F \rightarrow C$ that avoids transmission conflicts. $\square$

Based on the above mapping from multi-core stream allocation problem to the classical graph k -coloring problem, many classic algorithms can be applied for stream allocation. For example, in the case of $k = 2$, the graph coloring problem is equivalent to the graph bipartition problem, which can be solved in polynomial time. For the case of $k > 2$, many effective algorithms exist [16, 17] and we employ Z3 SMT solver to solve the multi-core stream allocation problem [18].

5 Performance Evaluation

We now present the evaluation results of our TSN ES design, which is implemented on an x86 platform consisting of a station powered by a 2.5GHz 11th Gen Intel i7 CPU paired with an Intel i225-T1 NIC. We also conduct extensive simulation-based experiments to validate the performance of the proposed multi-core stream allocation framework.

5.1 Experiment Setup

To evaluate the effectiveness of our ES design, we implement a Linux API-based ES for comparison, and both designs are deployed on the same hardware setup. Our test scenario involves two directly connected ESes. For this, we develop a listener using the Linux stack, incorporating `SO_TIMESTAMPING` for precise measurement of delay and jitter. This setup allows a fair comparison between the two designs by only alternating the talkers. For accurate delay measurement, we utilize the `phc2sys` [19] and DPDK example `ptp-client` programs to achieve sub-microsecond-level synchronization accuracy between the talker and the listener.

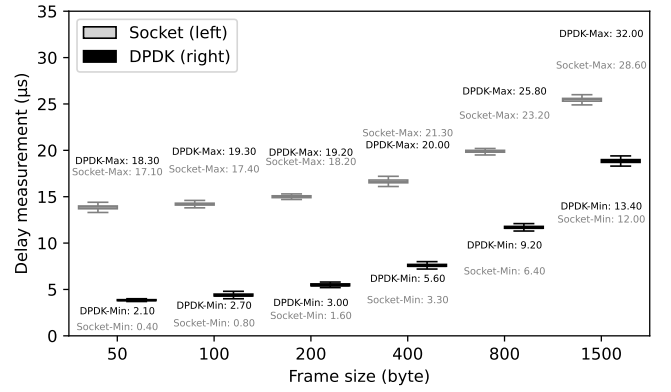


Figure 6. End-to-end delay measurement with various packet sizes.

5.2 End-to-End Delay

We first evaluate the end-to-end delay of the proposed ES design. We let the talker send a packet every 1 ms and record the time when CPU issues the sending command with the hardware timer on NIC. We also record the corresponding arrival time of the first bit of that packet on the listener. Since the two ESes are synchronized, we can calculate the end-to-end delay by subtracting the sending time from the arrival time. It is worth noting that since only one CPU core is used in this experiment, we do not enable the pre-fetch function.

Fig. 6 compares the end-to-end delay between our proposed DPDK-based ES and the Linux API-based ES with various packet sizes. The results show that, on average, our ES design can significantly reduce the delay compared to its Linux API-based counterpart. This improvement is largely attributed to DPDK’s user space implementation, which bypasses the kernel processing for lower latency, and its efficient memory access leveraging huge pages and NUMA support. However, outliers can be observed in both designs, where the maximum delay of our ES design occasionally exceeded that of the Linux API-based ES. We hypothesize that these anomalies might be due to measurement inaccuracies, potentially incurred from delays in the talker retrieving time from the NIC.

5.3 Throughput

We next present a comparative analysis of the throughput between the two designs. We configure the talker to send a single periodic stream with varying periods ranging from 5 to 50 μ s, with a step size of 5 μ and packet size 52 bytes. The packet loss ratio is recorded on the listener side. For testing the multi-core stream allocation framework, workloads are evenly distributed across two CPU cores. For instance, a stream with a 5 μ s period is divided into two parallel streams, each with a 10 μ s period, transmitted from two separate CPU cores.

Fig. 7(a) compares the throughput under varying traffic periods. Notably, the single-core DPDK-based design can efficiently handle traffic with a period of 35 μ s, outperforming the Linux API-based design which can only manage the traffic with a period of 40 μ s. More impressively, the multi-core DPDK framework can further handle the traffic with a period

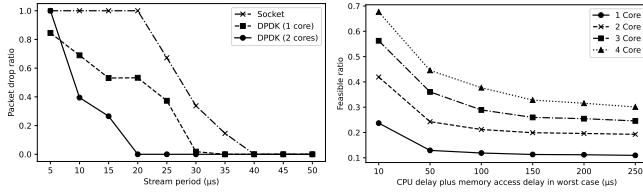


Figure 7. (a) Throughput comparison with varied traffic periods under different numbers of CPU cores. (b) The comparison of feasibility ratio under different numbers of CPU cores. (Core-0 used for synchronization is not counted)

of 20 μs , doubling the capacity of the single-core solution. Besides validating the maximum achievable throughput of our ES design with zero packet loss, our experiments also show that the proposed DPDK-based ES design can achieve even higher throughput when a certain level of packet loss is allowed. These results are omitted here due to the page limit.

5.4 Schedulability

Finally, we conduct simulation-based experiments to evaluate the schedulability of our multi-core stream allocation framework. Our focus is on the feasibility ratio, which represents the proportion of stream sets that can be successfully applied on the ES without contention. We begin the experiments by generating 10,000 random stream sets according to the settings outlined in [20]. These stream sets are then provided as input to a state-of-the-art TSN scheduling algorithm [21], to generate network-wide schedules. Based on the generated schedules, we assess how many of them can be realistically implemented on our ES, taking Constraint (1) into account.

Fig. 7(b) shows the impact of the number of available CPU cores on the feasibility ratio. It can be clearly observed from the results that the increasing number of available CPU cores leads to a higher feasibility ratio in our proposed multi-core parallel framework. Additionally, it is important to note that most existing scheduling algorithms, including the one referenced in [21], do not take these practical constraints into consideration. This highlights the need for developing a high-throughput TSN ES that are compatible with a broader range of existing scheduling algorithms.

6 Conclusion

In this study, we introduce an innovative software-based approach for TSN ES design, utilizing COTS hardware and DPDK open-source libraries to achieve high-precision, ultra-low latency and high throughput real-time packet transmissions for TSN applications. Our method is centered around the scheduler design and can effectively minimize CPU resource contention through the use of HPET and improve the precision by offloading the scheduling duties to the NIC. Additionally, our novel multi-core stream allocation framework ensures efficient management of real-time TSN traffic. Extensive testing in a real-world TSN testbed has demonstrated the superior performance of our approach over existing solutions, marking a significant advancement in TSN ES design.

This work not only tackles current challenges in TSN implementations but also provides a cost-effective, adaptable, and high-performance solution, contributing substantially to the field and paving the way for broader IIoT applications.

7 Acknowledgement

The work is supported in part by the NSF Grant CNS-1932480, CNS-2008463, CCF-2028875, CNS-1925706, and the NASA STRI Resilient Extraterrestrial Habitats Institute (RETHi) under grant number 80NSSC19K1076.

References

- [1] Norman Finn. 2018. Introduction to time-sensitive networking. *IEEE Communications Standards Magazine* 2, 2 (2018).
- [2] Gang Wang, Tianyu Zhang, Chuanyu Xue, Jiachen Wang, Mark Nixon, and Song Han. 2023. Time-Sensitive Networking for Industrial Automation: Challenges, Opportunities, and Directions. *arXiv preprint arXiv:2306.03691* (2023).
- [3] Wolfgang Fischer, Joseph Gelish, and Michael Hegarty. 2021. *Aerospace TSN use cases, traffic types, and requirements*.
- [4] TTEch Industrial. 2023. Edge IP solution.
- [5] Cisco. 2023. Industrial Ethernet 4000 Series Switches - Data Sheet.
- [6] Chenglong Li, Zonghui Li, Tao Li, Cunlu Li, and Baosheng Wang. 2023. A Deterministic Embedded End-system Tightly Coupled with TSN Schedule. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2023).
- [7] Wei Quan, Wenwen Fu, Jinli Yan, and Zhigang Sun. 2020. OpenTSN: an open-source project for time-sensitive networking system development. *CCF Transactions on Networking* 3 (2020).
- [8] Eleftherios Kyriakakis, Maja Lund, Luca Pezzarossa, Jens Sparsø, and Martin Schoeberl. 2020. A time-predictable open-source TTEthernet end-system. *Journal of Systems Architecture* (2020).
- [9] Chenglong Li, Tao Li, Junnan Li, Wenwen Fu, and Baosheng Wang. 2023. DRA: Ultra-Low Latency Network I/O for TSN Embedded End-Systems. In *31st International Symposium on Quality of Service (IWQoS)*.
- [10] James Coleman, Sara Almalih, Alexander Slota, and Yann-Hang Lee. 2019. Emerging cots architecture support for real-time TSN ethernet. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*.
- [11] Marcin Bosk, Filip Rezabek, Kilian Holzinger, Angela Gonzalez Marino, Abdoul Aziz Kane, Francesc Fons, Jörg Ott, and Georg Carle. 2022. Methodology and infrastructure for TSN-based reproducible network experiments. *IEEE Access* 10 (2022).
- [12] Christoph Lehr, Patrick Denzler, Thomas Frühwirth, and Wolfgang Kastner. 2023. Buffer Management for TSN-Enabled End Stations. In *19th International Conference on Factory Communication Systems (WFCS)*.
- [13] Linux Foundation. 2015. Data Plane Development Kit (DPDK).
- [14] Josef Dorr, Karl Weber, and Steven Zupuncic. 2018. *Use Cases IEC/IEEE 60802*.
- [15] Arnaldo Carvalho De Melo. 2010. The new Linux Perf tools. In *Slides from Linux Kongress*, Vol. 18.
- [16] Philippe Galinier, Alain Hertz, and Nicolas Zufferey. 2008. An adaptive memory algorithm for the k-coloring problem. *Discrete Applied Mathematics* 156, 2 (2008).
- [17] Alain Hertz, Matthieu Plumettaz, and Nicolas Zufferey. 2008. Variable space search for graph coloring. *Discrete Applied Mathematics* 156, 13 (2008).
- [18] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*.
- [19] Richard Cochran and Cristian Marinescu. 2010. Design and implementation of a PTP clock infrastructure for the Linux kernel. In *2010 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control and Communication*.
- [20] Chuanyu Xue, Tianyu Zhang, Yuanbin Zhou, Mark Nixon, Andrew Loveless, and Song Han. 2024. Real-Time Scheduling for 802.1Qbv Time-Sensitive Networking (TSN): A Systematic Review and Experimental Study. In *Proceedings of the 30th IEEE Real-Time and Embedded Technology and Applications Symposium*.
- [21] Daniel Bujosa, Mohammad Ashjaei, Alessandro V Papadopoulos, Thomas Nolte, and Julián Proenza. 2022. HERMES: Heuristic multi-queue scheduler for TSN time-triggered traffic with zero reception jitter capabilities. In *Proceedings of the 30th International Conference on Real-Time Networks and Systems*.